

DBIS PROJECT

Vaka Sri Varshitha - 23b1030, Pranathi Sreeja - 23b0936, Boyina Sanjana - 23b0976 ,
Kokkiligadda Manaswi - 23b1046

1 Project Report: Development, Motivation, and Methodology

1.1 Motivation

Modern applications often experience read-heavy workloads, where a small subset of data is accessed repeatedly. To reduce latency and database load, systems commonly introduce external caches like Redis or Memcached. While effective, this approach complicates system design by forcing developers to manage two separate systems a cache and a database each with different consistency models, query interfaces, and failure modes.

This dual-layer architecture introduces several challenges:

- Cache invalidation complexity, often leading to stale or inconsistent data
- Increased application logic, including manual cache lookups and write-through policies
- Operational overhead of maintaining and scaling an additional service
- Mismatch in query capabilities, as key-value stores lack the expressiveness of SQL

To address these issues, this project explores an alternative design where caching is integrated directly into PostgreSQL. By using a local PostgreSQL instance as a cache, the system preserves the SQL interface, eliminates the need for application-level cache management, and leverages built-in mechanisms like logical replication for maintaining data freshness.

The key idea is to make caching transparent to the application: queries are automatically routed, cached, or fetched from the master without requiring any changes to existing code. This simplifies development while still achieving the performance benefits of caching and maintaining strong consistency for writes.

1.2 What Was Implemented

The project resulted in a PostgreSQL extension called `pg_cache_router` that integrates directly into the query lifecycle.

1.2.1 1. Query Interception via Hooks

Two PostgreSQL hooks were implemented:

- **planner_hook**: Used to intercept and rewrite queries before execution planning
- **ExecutorStart_hook**: Used to implement runtime cache logic for SELECT queries

These hooks allowed full control over query routing decisions.

1.2.2 2. Cache Routing Logic

The system classifies queries into three categories:

- Primary-key SELECT queries (cacheable)
- Complex SELECT queries (routed to master)
- Write queries (always routed to master)

A detection mechanism was implemented to identify queries of the form:

```
SELECT * FROM table WHERE primary_key = <value>
```

1.2.3 3. Read-Through Cache Implementation

For cacheable queries:

- Cache HIT: serve data from local table
- Cache MISS:
 - Fetch data from master using `dblink`
 - Insert into local cache
 - Return result

1.2.4 4. LRU Cache Metadata

A metadata table `cache_lru_meta` was implemented to track:

- Table name
- Primary key value
- Last access timestamp

This enables LRU-based eviction policies implemented in SQL functions.

1.2.5 5. Write Routing and Cache Invalidation

All write operations (INSERT, UPDATE, DELETE):

- Are forwarded to the master database using `dblink.exec`
- Trigger cache invalidation:
 - Delete cached rows for the table
 - Clear metadata entries

1.2.6 6. Query Rewriting

Queries routed to master are rewritten into:

```
SELECT * FROM dblink(connection, query) AS t(schema)
```

This ensures compatibility with PostgreSQL's execution model.

1.2.7 7. Build and Deployment

The system was deployed using two PostgreSQL clusters to simulate a master-cache architecture on a single machine.

PostgreSQL Build from Source

We followed postgresql documentation for installing source code

Cluster Initialization

```
# Master
initdb -D /usr/local/pgsql/data
# Cache
initdb -D /usr/local/pgsql/data2
```

Configuration

- Master: port = 5432
- Cache: port = 5433

Extension Setup

```
make
sudo make install
shared_preload_libraries = 'pg_cache_router'
```

1.3 3. Code Implementation Details

1.3.1 Execution Flow of the System

The working of the extension can be understood as a step-by-step flow through PostgreSQL's query lifecycle:

Step 1: Extension Initialization When PostgreSQL starts, the function `_PG_init()` is executed. This registers the `planner_hook` and `ExecutorStart_hook`, allowing the extension to intercept all incoming queries.

Step 2: Query Planning Phase When a query is issued, PostgreSQL first invokes `cache_router_planner()`.

- The query is analyzed to determine its type (SELECT, INSERT, UPDATE, DELETE)
- `is_pk_equality_select_query()` checks if it is a primary-key equality query
- If it is a complex query or belongs to a master-routed table:
 - The query is rewritten using `rewrite_to_dblink()`
 - Execution is redirected to the master database
- For write queries:
 - `rewrite_write_to_dblink_exec()` or `rewrite_write_returning_to_dblink()` is used

Step 3: Execution Phase Before execution begins, `cache_router_ExecutorStart()` is invoked.

- `is_pk_equality_select()` verifies again if the query is cacheable
- `try_serve_from_cache()` checks whether the requested row exists locally

Step 4: Cache Hit If the row is found in cache:

- The cached data is returned directly
- LRU metadata is updated to reflect recent access

Step 5: Cache Miss If the row is not found:

- `populate_cache_from_master()` fetches the row using dblink
- The row is inserted into the local cache
- The query is rewritten via `rewrite_to_dblink()` to return the result

Step 6: Write Operations For INSERT, UPDATE, and DELETE queries:

- Queries are always executed on the master database
- Cache is invalidated to maintain consistency

Step 7: Schema Handling When rewriting queries using dblink:

- `get_targetlist_schema_for_dblink()` builds the required output schema
- `get_table_schema_for_dblink()` retrieves schema dynamically
- `_PG_init()` — initializes extension and registers hooks
- `_PG_fini()` — restores original PostgreSQL hooks
- `cache_router_planner()` — decides query routing during planning
- `cache_router_ExecutorStart()` — executes cache hit/miss logic
- `is_pk_equality_select()` — detects primary-key queries from execution plan
- `is_pk_equality_select_query()` — detects cacheable queries from parsed query
- `try_serve_from_cache()` — checks cache and updates LRU metadata
- `populate_cache_from_master()` — fetches missing rows using dblink
- `rewrite_to_dblink()` — rewrites SELECT queries to execute on master
- `rewrite_write_to_dblink_exec()` — forwards write queries
- `rewrite_write_returning_to_dblink()` — handles write queries with RETURNING
- `get_targetlist_schema_for_dblink()` — builds schema for dblink results
- `get_table_schema_for_dblink()` — fetches schema dynamically

1.3.2 4. Performance Observation

- Compared execution time for repeated PK queries
- Observed reduced latency for cache hits
- Verified that complex queries still execute correctly via master
- To evaluate the effectiveness of the caching mechanism, a workload of 200 queries was executed under 2 cases, one with 90% cache hit rate and other with 100% cache hit rate, the times were around **0.5sec** and **0.3sec** respectively

1.4 Use of AI Tools

1.4.1 How AI Was Used

- To understand PostgreSQL internal structures such as:
- To debug issues
- To refine logic for query classification and rewriting

1.4.2 Prompts Used

"Explain how PostgreSQL planner hook works internally"

"How to extract WHERE clause from PlannedStmt in Postgres?"

"Why does IndexScan not always have indexqual?"

"How to safely use SPI in PostgreSQL extensions?"